

AWD-COORD: Autonomous Multi-Node Alternate Wetting and Drying Irrigation Coordinator

Sadid Ahmed

Dept. of CSE, UIU

ID: 0112330154

sahmed2330154@bscse.uiu.ac.bd

M.M. Sayem Prodhan

Dept. of CSE, UIU

ID: 0112330411

mprodhan2330411@bscse.uiu.ac.bd

Mostafizur Rahman

Dept. of CSE, UIU

ID: 0112330844

mrahman2330844@bscse.uiu.ac.bd

Saikat Raihan

Dept. of CSE, UIU

ID: 0112230611

sraihan223611@bscse.uiu.ac.bd

Fahad Parvez Sagor

Dept. of CSE, UIU

ID: 0112330829

fsagor2330829@bscse.uiu.ac.bd

Abstract—Continuous flooding, the traditional irrigation method for rice cultivation, drives heavy groundwater extraction and contributes to methane emissions from waterlogged soil. Alternate Wetting and Drying (AWD) is an established, BRRI/IRRI-documented technique that reduces water use without a significant yield penalty, yet real-world adoption remains rare. This is not because AWD itself is unproven, but because smallholder farmers who share a single irrigation pump across several plots have no reliable way to coordinate whose plot is watered, when, and for how long. A sensor that simply reports “your field is dry” does not solve this: someone still has to negotiate turns manually.

AWD-COORD is a distributed, rule-based coordination layer that automates this scheduling problem. A network of ESP32-based field nodes reports soil and water-depth conditions over ESP-NOW to a bridge node (the Coordinator Proxy), which relays this data over USB serial to a Python-based scheduler running on a laptop. The scheduler applies a priority queue that weighs depth deficit and wait time to decide which plot is serviced next, issuing valve and pump commands back through the same path. A Shadow Coordinator ESP32 mirrors system state and enforces a safe shutdown (all valves closed, pump off) if the primary coordination path is lost, without attempting to replicate scheduling intelligence it does not possess.

The system is explicitly rule-based; no machine learning is used or planned. As of this report, the four field nodes and the pump/gateway node are physically built and individually verified on real hardware, and production ESP-NOW firmware for all board types has been written and is undergoing first integration testing. The Python coordinator’s core decision logic (start/stop sequencing, fault handling, override respect, and command reconciliation against reported hardware state) has been implemented and is covered by an automated unit-test suite, and is being validated against live hardware in a dry-run mode that computes but does not yet dispatch commands. This report documents the system’s architecture, current verified state, and a bench validation plan; it deliberately does not present fabricated experimental results, since end-to-end hardware validation is still in progress at the time of writing.

I. INTRODUCTION AND BACKGROUND

A. Continuous flooding versus Alternate Wetting and Drying

Traditional rice cultivation keeps paddies continuously flooded from transplanting through most of the growing season. This practice is water-intensive and is a documented

driver of groundwater depletion in water-stressed agricultural regions. Alternate Wetting and Drying (AWD) is a cultivation technique, documented by BRRI and IRRI [1], [2], in which the field is allowed to dry to a defined depth below the soil surface before re-flooding, rather than being kept permanently submerged. Published agronomic sources report substantial water savings from AWD [3] without a significant yield penalty when growth-stage-sensitive windows (notably flowering) are excluded from drying cycles.

AWD itself is not the unsolved engineering problem this project addresses. The technique is established and does not require automation to exist as a farming practice.

B. The coordination problem

The unsolved problem is coordination among farmers who share a single pump across a small command area. AWD requires precise, plot-specific timing: each plot must be monitored individually and watered only when its own depth crosses the threshold, growth stage permits it, and it is fairly next in a shared queue. Where several farmers share one pump, there is no reliable way to track whose plot needs water, enforce fair turns, or act on real-time soil conditions without a system to do it for them. A simple sensor dashboard that reports field conditions is insufficient on its own, because someone would still need to manually interpret the readings, negotiate access to the shared pump, and issue commands — the coordination step, not the sensing step, is what is missing.

C. Objectives

- Automate multi-plot AWD scheduling across several farmers sharing a single pump, without requiring manual negotiation.
- Implement rule-based (not AI-based) fairness among plots, so no single plot can monopolize the shared pump.
- Provide a safe failure mode: if the coordination path fails, the system must default to a safe state rather than an unmonitored one.

- Keep the solution affordable and appropriate for smallholder deployment, evaluating every component by functional value rather than technical sophistication.
- Demonstrate the coordination mechanism on a tabletop testbed as a feature showcase, while being explicit that this is not a field-deployed system.

II. SYSTEM ARCHITECTURE AND HARDWARE DESIGN

A. Architectural overview

The system consists of seven ESP32 microcontroller boards organized around a single coordination path, plus a laptop running the decision-making software. Communication between boards uses ESP-NOW, a connectionless, low-latency protocol built on the ESP32's WiFi radio [4]; the laptop is reached over a single USB serial link to one designated bridge board. Figure 1 shows the overall topology.

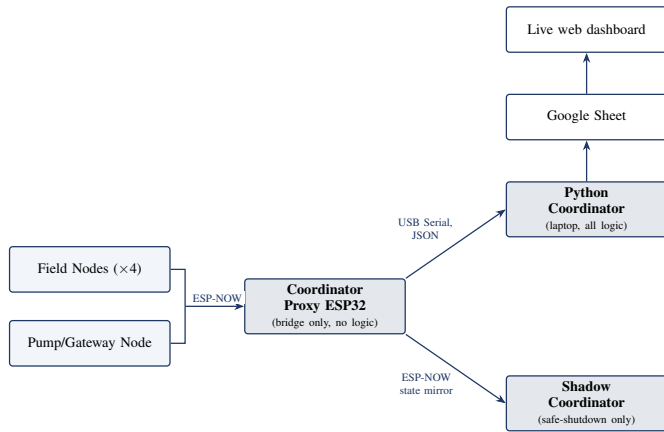


Fig. 1. AWD-COORD system architecture. The Coordinator Proxy is a bridge only; all scheduling logic runs in the Python Coordinator. The Shadow Coordinator is limited to safe shutdown on loss of the primary path.

A deliberate architectural distinction runs through this design: the Coordinator Proxy ESP32 is a bridge only. It relays ESP-NOW packets to and from the laptop over USB serial, and separately hosts a SIM800L GSM module for real SMS status queries; it holds no scheduling logic of its own. All decision-making — the priority queue, fairness enforcement, fault handling, and command sequencing — runs in the Python Coordinator on the laptop. This separation was intentional: it keeps the resource-constrained microcontroller layer simple and moves all algorithmic complexity to a platform where it can be tested, versioned, and modified without reflashing hardware.

B. Deliberate exclusions from the architecture

A Raspberry Pi 5 (8 GB RAM, with cooling and camera) was made available to the team by the university at no cost and was deliberately excluded from the final design. The coordination workload — a priority-queue calculation over a handful of small sensor values every few seconds — does not require that level of compute; an ESP32 network plus a laptop already handles it. Since the project's premise is

affordability for smallholder farmers, and a real farmer would bear the full retail cost of a Pi-class computer, including it would have undercut the project's own cost-to-value argument. No machine learning or AI inference is used anywhere in the scheduling logic; all decisions are deterministic and rule-based.

C. Hardware component specification

TABLE I
HARDWARE COMPONENT SPECIFICATION

Component	Role	Qty
ESP32-DevKitC	Main controller (4 field, 1 pump, 1 proxy, 1 shadow)	7
Depth potentiometer	Simulates perched water depth (0–20 cm)	4
Crop-stage potentiometer	Rotary substitute; 3 zones (vegetative/flowering/ripening)	4
Soil moisture sensor	Secondary/cross-check signal	4
NPN transistor probe	Water-presence detection (polarity: HIGH = water detected)	4
SG90 servo	Valve actuator (0° closed, 90° open)	4
Manual override switch	Farmer bypass of automation	4
L298N motor driver	Drives the shared pump (Channel B)	1
Mini submersible pump	Shared irrigation pump (on/off control)	1
INA219 current sensor	Pump power monitoring	1
SSD1306 OLED	Pump node status display	1
Rain drop sensor	Detects rainfall to pause irrigation	1
SIM800L GSM module	Real SMS status interface on the Proxy	1
Solar panel + Li-ion cell	Solar charging for the pump node	1

D. Verified pin assignments

The following pin maps are confirmed on physical hardware and are treated as authoritative over any earlier design documents in this project's history.

- 1) *Field node (identical wiring across all four boards):*
- 2) *Pump / gateway node:*
- 3) *Coordinator proxy and shadow coordinator:*

E. Communication topology

The network uses a star-like ESP-NOW topology in which the Coordinator Proxy is the unicast peer registered by every field node and the pump node; the Proxy in turn holds all seven boards as registered peers. This was a deliberate design choice over broadcast messaging, to keep traffic explicitly addressed and avoid field nodes observing each other's data. ESP-NOW is reliable at roughly 150–220 m in open conditions, degrading to 40–100 m with obstructions, and has no native multi-hop relay. This is adequate for the tabletop demonstration, where all nodes are within roughly one meter of each other, but is explicitly not representative of a real, spread-out farm command area; a documented real-world scaling path (ESP-NOW long-range mode, or a clustered LoRa backbone) exists but is out of scope for the current build.

TABLE II
FIELD NODE PIN ASSIGNMENTS

Signal	GPIO	Notes
Depth potentiometer (wiper)	32	ADC1
Crop-stage potentiometer (wiper)	33	ADC1; < 1365 vegetative, < 2730 flowering, else ripening
Soil moisture (analog)	34	ADC1, input-only pin
Water-presence transistor (collector)	35	Input-only pin, external pull-up required. HIGH = water detected (hardware-verified; corrects an earlier documentation error stating the inverse)
Servo signal	25	90° = valve open, 0° = valve closed
Green LED	26	Solid = online, automatic mode
Red LED	27	On = manual override engaged
Manual override switch	14	INPUT_PULLUP, LOW = engaged. GPIO12 deliberately avoided (boot-strapping risk)

TABLE III
FIELD NODE PIN ASSIGNMENTS

Signal	GPIO	Notes
I ² C SDA / SCL	21 / 22	Shared bus for INA219 and OLED, 100 kHz, 50 ms timeout
Motor driver IN3 (MOTOR_IN3)	26	Digital control for L298N Channel B
Motor driver IN4 (MOTOR_IN4)	27	Digital control for L298N Channel B
Rain sensor (analog)	33	ADC1
Failure-simulation button	25	INPUT_PULLUP, LOW = engaged; forces the pump off and reports a fault flag. This pin was originally a manual pump on/off switch during bench testing; that use is retired in production firmware

TABLE IV
COORDINATOR PROXY AND SHADOW COORDINATOR PIN ASSIGNMENTS

Board	Signal	GPIO	Notes
Coordinator Proxy	USB Serial to laptop	1 / 3 (fixed)	Carries newline-delimited JSON at 115200 baud ; no SMS handling implemented in current firmware
Coordinator Proxy	SIM800L UART	16 / 17	Off = no sync yet, solid = primary healthy, blinking = failover active
Shadow Coordinator	Status LED	4	

III. SOFTWARE ARCHITECTURE AND FIRMWARE IMPLEMENTATION

A. Division of responsibility

Firmware on each ESP32 board is responsible only for reading its own sensors, actuating its own hardware, and relaying fixed-format ESP-NOW packets. All scheduling intelligence lives in a single Python process, the Coordinator, running on a laptop and connected to the Coordinator Proxy over USB serial using a newline-delimited JSON protocol. No AI or machine learning is used anywhere in this pipeline; every decision is a deterministic, auditable rule.

B. Finite state machine

The Coordinator’s scheduling loop is implemented as a finite state machine with four states. This supersedes an earlier five-state design (which additionally proposed a distinct REQUEST_QUEUE state and treated manual override as a global mode); the current, implemented design instead treats override as a per-plot flag that is checked within every state, since override is farmer-controlled and can occur independently of the shared pump’s overall condition.

TABLE V
COORDINATOR FINITE STATE MACHINE

State	Meaning	Entry condition	Exit condition
IDLE	No plot is currently being irrigated; the system evaluates eligibility each tick	Startup, or completion/abort of a previous cycle	An eligible plot is selected
ARMING	A valve-open command has been sent; awaiting confirmation that the valve has actually opened before starting the pump	A plot is selected from IDLE	Confirmed open (pump turned on) or timeout/abort (return to IDLE)
ACTIVE	The pump is running and the valve is confirmed open; irrigation in progress	Valve-open confirmed in ARMING	Target depth reached, timeout elapsed, or an abort condition (offline node, override engaged, no water detected)
FAULT	The pump has reported a fault condition (failure-simulation button engaged)	pump.fault = true, at any time	Fault condition clears

C. Priority scheduling algorithm

This supersedes an earlier proposed First-Come-First-Served approach with threshold overrides. The implemented algorithm is a weighted priority score, computed for every eligible plot on each scheduling tick, so that the neediest and

longest-waiting plot is serviced first rather than plots being serviced strictly in arrival order:

$$\begin{aligned} \text{score}(\text{plot}) = & (\text{depth_deficit} \times W_{\text{depth}}) \\ & + (\text{seconds_since_last_service} \times W_{\text{wait}}) \\ & - (\text{equity_penalty} \times W_{\text{equity}}) \end{aligned} \quad (1)$$

where `depth_deficit` is the shortfall below the AWD irrigation threshold, `seconds_since_last_service` is the time elapsed since that plot was last irrigated, and the `equity_penalty` term reserves capacity for a fairness mechanism that caps any single plot's cumulative share of pump time. A plot is eligible for scheduling only if it is online, reporting valid sensor data, not under manual override, and below the irrigation threshold depth.

D. Safety invariants

The scheduler enforces the following invariants on every decision cycle, verified by an automated unit-test suite:

- The pump is never turned on until its corresponding valve-open command has been confirmed by the field node's reported state, not merely assumed from timing.
- When stopping irrigation, the pump is always commanded off before the valve is commanded closed.
- No command is ever sent to a plot with an active manual-override flag.
- Pump-on commands are blocked while a fault condition is active; pump-off commands remain permitted at all times, since turning the pump off is always the safe action.
- On startup, or if the serial link to the hardware is lost, the system does not attempt to command hardware it cannot reliably reach, and returns to a safe idle state.
- Each scheduling tick reconciles the system's intended state against the hardware's self-reported state (reported valve position and reported pump state), and reissues commands if a discrepancy is detected, to guard against dropped ESP-NOW packets.

E. Shadow Coordinator failover

A second ESP32, the Shadow Coordinator, passively receives a periodic state-mirroring packet from the Coordinator Proxy. If this packet is not received within a defined timeout, the Shadow Coordinator concludes the primary coordination path (the laptop, the Proxy, or the link between them) has failed, and independently issues a one-time safe-shutdown command directly to all field nodes and the pump node: every valve closed, pump off. This design deliberately does not attempt to replicate the Python Coordinator's scheduling intelligence during failover, since that logic exists only on the laptop; the Shadow Coordinator's role is limited to guaranteeing a safe state, not continued operation.

F. Communication protocol

ESP-NOW packets between boards are fixed-size binary structures sharing a one-byte type discriminator. The Coordinator Proxy translates these into newline-delimited JSON

objects over USB serial for consumption by the Python Coordinator, and translates JSON commands from the Coordinator back into the corresponding binary ESP-NOW packets.

TABLE VI
COMMUNICATION PROTOCOL

Packet / message	Direction	Key fields
FIELD_STATUS	Field node → Proxy → laptop (JSON)	node id, depth, crop stage, soil moisture, water present, override active, valve open (commanded state), uptime
PUMP_STATUS	Pump node → Proxy → laptop (JSON)	pump on, rain reading, voltage, current, power, fault flag, uptime
HEARTBEAT	Any node → Proxy	node id; three missed heartbeats (approx. 6 s) marks a node offline
VALVE_COMMAND / pump command (JSON)	Laptop → Proxy → field or pump node	node id and open/close, or pump on/off
SHADOW_SYNC	Proxy → Shadow Coordinator	per-node online flags, pump state, active plot, timestamp

G. Telemetry and dashboard architecture

The intended telemetry path writes the Coordinator's live state to a Google Sheet at a fixed interval via a service-account-authenticated API, with the sheet published to the web as CSV. A static HTML/CSS/JavaScript web dashboard, hosted with no backend server, polls the published CSV on an interval and renders plot status, pump status, and system state. The dashboard is a monitoring display only and has no control authority over the physical system; all control originates from the Python Coordinator.

IV. ECONOMIC AND ENVIRONMENTAL CONTEXT

A. Water savings

Published AWD literature [1], [3] reports water savings in the range commonly cited as 25–50% relative to continuous flooding, without a significant yield penalty when flowering-stage drying is avoided. The general form of the water-savings calculation used in agronomic literature is:

$$W_{\text{saved}} = \frac{V_{\text{CF}} - V_{\text{AWD}}}{V_{\text{CF}}} \times 100\% \quad (2)$$

where V_{CF} is the water volume applied under continuous flooding and V_{AWD} is the water volume applied under AWD for a comparable plot and season. AWD-COORD does not itself alter the water savings achievable by the AWD technique; its contribution is enabling multiple farmers to practice AWD reliably despite sharing a single pump, which published sources identify as an adoption barrier independent of the technique's proven efficacy.

B. Methane emission reduction

Waterlogged, continuously flooded soil creates anaerobic conditions that favor methanogenic bacteria, making flooded rice paddies a documented source of methane emissions. AWD's periodic soil aeration is reported in agronomic literature [2] to reduce methane emissions from rice cultivation, since the technique interrupts the anaerobic conditions methane-producing bacteria require.

C. Cost-to-value philosophy

Every hardware and feature decision in this project was evaluated against a stated cost-to-value principle: impact to the farmer or command area weighed against added hardware cost, complexity, and maintenance burden, rather than technical sophistication alone. This is illustrated concretely by three decisions made during the build: the exclusion of a free Raspberry Pi 5 because the coordination workload does not require it and a real farmer would bear its full cost; the substitution of a transistor-based water-presence probe for a costlier YF-S201 flow sensor; and the choice of a real SIM800L SMS module (offering access for farmers without smartphones) over a smartphone-app-only interface.

V. CHALLENGES, RISK MITIGATION, AND SAFETY

A. Wireless interference and range in the 2.4 GHz band

ESP-NOW operates on the same 2.4 GHz ISM band as WiFi and Bluetooth, and is subject to the same interference and range limitations. Plain ESP-NOW without multi-hop relay is documented as reliable to roughly 150–220 m in open conditions and 40–100 m with obstructions. This is adequate for the tabletop demonstration but is explicitly not sufficient for a real, spread-out farm command area. Two real-world scaling paths are documented for any future continuation of this project beyond the competition context: ESP-NOW's long-range mode, or a clustered architecture in which small ESP-NOW groups each report to a local gateway that relays over LoRa to a central coordinator.

B. Sensor durability in a wet environment

Sensors exposed to standing water (the water-presence probe, and any submerged wiring) are a corrosion risk in sustained deployment, though this is a lesser concern for the tabletop demonstration's short operating duration. The water-presence probe's exposed-contact design was chosen specifically because a full flow sensor was judged too costly for the project's affordability goal; a production deployment would need to evaluate corrosion-resistant probe materials or periodic replacement as a maintenance cost.

C. Pump driver hardware fault and mitigation

During early bench integration, the originally selected L9110S motor driver was found to have a partially degraded output channel that leaked voltage, failing to fully de-energize the pump when commanded off. While an initial firmware workaround (engaging electronic brake mode) was attempted, the fundamental unreliability of the driver at the required

current levels prompted a full hardware migration. The pump node was successfully rewired to use an L298N motor driver (utilizing Channel B via IN3 and IN4). This migration resolved the voltage leakage, provided superior thermal performance, and eliminated the nonzero resting current artifact introduced by the previous software brake workaround.

D. Manual override and pump dry-running interlocks

The scheduler enforces that no valve or pump command can ever reach a plot under active manual override, in every code path including startup safety flush, fault handling, and mid-irrigation abort — verified by unit tests covering each of these paths independently. Additionally, the scheduler never commands the pump on until the corresponding field node has reported its valve as actually open (not merely assumed from elapsed time alone, with a time-based fallback only when a node's valve-state field is unavailable), and continuously reconciles its intended state against hardware-reported state each cycle to catch commands lost to packet drops. These mechanisms are directed specifically at preventing the pump from running against a closed valve, which is the primary dry-running risk in this architecture.

E. Coordinator process failure versus link failure

The Shadow Coordinator's failover mechanism protects against loss of the communication link between the Coordinator Proxy and the laptop, or loss of the Proxy itself, by watching for the periodic state-mirroring packet. It does not, in the current design, protect against the specific case where the Python Coordinator process itself crashes while the Proxy remains powered and connected, since the Proxy would have no reason to stop relaying data. A planned mitigation, not yet implemented, is a laptop-heartbeat requirement on the Proxy: if the Proxy does not receive a periodic acknowledgment from the running Python process within a timeout, it independently withholds further commands and signals the Shadow Coordinator, closing this specific gap.

VI. CONCLUSION AND FUTURE WORK

A. Summary of current achievements

- A complete seven-node hardware architecture designed, with four field nodes and the pump/gateway node physically built and individually verified on real hardware.
- A documented, deliberate separation between a bridge-only Coordinator Proxy and a fully rule-based Python Coordinator holding all scheduling intelligence.
- A four-state scheduler finite state machine with six explicit safety invariants, implemented and covered by an automated unit-test suite.
- A Shadow Coordinator failover design that defaults to a safe, fully-closed state rather than attempting unsupported autonomous scheduling.
- A field-discovered pump driver hardware fault diagnosed and mitigated in firmware, with the fix independently verified against real commands.

- A defined bench verification plan covering timing, range, latency, and fault-recovery behavior, to be executed as hardware integration completes.

B. Future enhancements

- Complete the bench verification plan (Section ??) and populate this report, or a follow-up report, with real measured results.
- Implement the remaining designed scheduler rules: fairness/equity enforcement, growth-stage suspension, calibrated rain-hold, and predictive scheduling based on observed depth-drop rate.
- Close the Python-process-crash gap identified in Section V-E with a laptop-heartbeat mechanism on the Coordinator Proxy.
- For any deployment beyond the competition tabletop context, migrate the communication layer to ESP-NOW long-range mode or a clustered LoRa backbone, as the current design's range is explicitly only adequate for a tabletop demonstration.
- Evaluate solar and battery performance under real outdoor charging conditions, since this has been wired but not load-tested.
- Investigate corrosion-resistant water-presence sensing for any deployment lasting a full growing season.

REFERENCES

- [1] R. M. Lampayan, R. M. Rejesus, G. R. Singleton, and B. A. Bouman, "Adoption and economics of alternate wetting and drying water management for irrigated lowland rice," *Field Crops Research*, vol. 170, pp. 95–108, 2015.
- [2] M. Richards and B. Sander, "Alternate wetting and drying in irrigated rice," *Climate-Smart Agriculture Practice Brief*, vol. 2, 2014.
- [3] B. Bouman and T. Tuong, "Field water management to save water and increase its productivity in irrigated lowland rice," *Agricultural Water Management*, vol. 49, no. 1, pp. 11–30, 2001.
- [4] M. Ali and et al., "Iot-based smart irrigation system using esp32 and esp-now," in *2021 IEEE International Conference on Smart Agriculture (ICSA)*. IEEE, 2021, pp. 1–6.